

Project Implementation Plan

WEB-BASED BUSINESS MANAGEMENT SYSTEM

3. Project Implementation Plan

The Business Management System will be developed using an incremental approach, where each phase builds upon the previous one. This approach allows continuous testing and validation while reducing project risks.

PHASE	ACTIVITIES	EXPECTED DURATION	POTENTIAL RISKS	MITIGATION STRATEGY
1. Project Planning & Requirements Analysis	Define project objectives, gather functional and non-functional requirements, identify users, define project scope, prepare use cases.	Week 1	Incomplete or changing requirements.	Conduct thorough requirements analysis and validate requirements before implementation.
2. System Design	Design use case diagrams, conceptual ERD, logical ERD, physical ERD, database architecture, and overall system architecture.	Week 2	Poor system design leading to redesign later.	Review and validate all diagrams before implementation begins.
3. Database Development	Create PostgreSQL database, implement tables, relationships, constraints, indexes, triggers, functions, and seed data.	Week 3	Database design errors, data inconsistency, foreign key conflicts.	Test each database module incrementally and perform regular database validation.
4. Backend Development	Develop REST APIs using NestJS, implement authentication, RBAC, product, customer, supplier, sales, purchase order, payment, and reporting modules.	Weeks 4–6	Learning curve with NestJS and API integration issues.	Follow modular development, consult official documentation, and test APIs continuously.
5. Frontend Development	Develop responsive user interfaces using Next.js, TypeScript, and Tailwind CSS for all system modules.	Weeks 7–8	User interface complexity and inconsistent user experience.	Build reusable components and conduct regular usability testing.

PHASE	ACTIVITIES	EXPECTED DURATION	POTENTIAL RISKS	MITIGATION STRATEGY
6. System Integration	Integrate frontend with backend APIs, connect all modules, and verify complete system functionality.	Week 9	Integration failures between frontend and backend.	Perform integration testing after completing each module rather than waiting until the end.
7. Testing & Debugging	Conduct unit testing, integration testing, system testing, and fix identified defects.	Week 10	Undetected software defects and performance issues.	Test each module thoroughly, maintain test cases, and resolve defects promptly.
8. Documentation & Deployment	Prepare technical documentation, user manual, final report, deploy the application, and perform final demonstrations.	Week 11	Documentation delays or deployment failures.	Maintain documentation throughout development and perform deployment in a controlled test environment before production.

Project Development Roadmap



Deliverables by Phase

PHASE	DELIVERABLE
Planning	Project Proposal, Objectives, Requirements Specification
Design	Use Case Diagram, Conceptual ERD, Logical ERD, Physical ERD
Database	PostgreSQL Database Schema, SQL Scripts
Backend	RESTful API (NestJS)
Frontend	Responsive Web Application (Next.js)
Integration	Fully Integrated System
Testing	Test Report and Bug Fixes
Documentation	User Manual, Technical Documentation, Final Project Report

Overall Risk Summary

RISK	IMPACT	MITIGATION STRATEGY
Changing project requirements	Delays and redesign	Finalize requirements before implementation and document any approved changes.
Learning new technologies (NestJS, PostgreSQL, Prisma)	Slower development	Allocate time for learning, use official documentation, and build small prototypes before implementing major modules.
Database design errors	Data inconsistency and application failures	Validate ERDs, normalize the database, and test relationships incrementally.
Integration challenges	Modules fail to communicate	Develop and test APIs independently before frontend integration.
Time constraints	Incomplete project	Prioritize core features (MVP) and defer optional enhancements to future versions.
Data loss during development	Loss of progress	Use Git for version control and perform regular PostgreSQL database backups.
Security vulnerabilities	Unauthorized access or data compromise	Implement secure authentication, role-based access control, password hashing, and input validation.
Performance issues	Slow response times	Optimize SQL queries, use indexes, and perform performance testing before deployment.